

Software Requirements Specification

CMSI 4072 | Spring 2026

5.1 Introduction

Sammy's Games is a browser-based software application that provides users with a centralized platform for playing original board and card games. The system consists of:

- User Interface (UI): Handles user interaction.
- Game Hub Module: Provides a centralized game selection interface.
- Game Modules: Self-contained files implementing specific games, each managing its own game state.
- AI Opponent Module: Optional module for computer-controlled gameplay.

The system operates entirely within a modern web browser and requires no additional software installation.

High-Level System Architecture (Component Diagram):

User Interface -> Game Hub -> Game Module <- AI Opponent Module

Component Relationships:

- User Interface: Communicates with the Game Hub and active Game Modules.
- Game Hub: Initializes the selected Game Module.
- Game Modules: Self-contained and manage their own game state.
- AI Opponent Module: Interacts with the active Game Module during computer-controlled turns.

The table below shows the internal submodules within each component and their responsibilities:

Component	Submodule	Responsibility
User Interface	Event Handler	Captures and routes user input
User Interface	Renderer	Updates the display based on current game state
User Interface	Navigation Controller	Manages transitions between the Hub and Game Modules
Game Hub Module	Game Registry	Maintains the list of available games
Game Hub Module	Loader	Initializes the selected Game Module

Game Module (each)	State Manager	Owns and updates the game state
Game Module (each)	Rule Engine	Validates moves and detects end conditions
AI Opponent Module	Move Generator	Enumerates valid moves from the current state
AI Opponent Module	Evaluator	Selects the best move to play

Interface definitions between components:

- Game Hub passes a game identifier to the Loader, which initializes the corresponding Game Module.
- Game Modules expose the current state to the UI Renderer after each action.
- The AI Opponent Module reads the current state from the Game Module and submits a move back to it.

The remainder of this document is structured as follows. Section 5.2 contains the Functional Requirements. Section 5.3 contains the Performance Requirements. Section 5.4 contains the Environment Requirements.

5.2 Functional Requirements

Functional requirements define observable behaviors and capabilities that the system shall provide. They describe what the system must do without specifying how it will be implemented. Each requirement is written in a "shall" statement and is testable.

5.2.1 Game Hub Interface

The Game Hub provides a centralized interface that allows users to view and select available games.

Requirements:

5.2.1.1 The system shall display a main menu listing all games included in the current release.

The list will include the name of each game.

If the game list fails to load, the system will display an error message rather than an empty menu.

5.2.1.2 The system shall allow the user to select exactly one game from the main menu.

5.2.1.3 The system shall load the selected game interface after the user selects a game.

5.2.1.4 The system shall display a visible "Return to Main Menu" control on every game screen.

5.2.1.5 The system shall return the user to the main menu when the "Return to Main Menu" control is activated.

5.2.2 Game Rule Presentation

Each game must clearly communicate its rules and objectives.

Requirements:

5.2.2.1 The system shall display the rules of the selected game before gameplay begins.

5.2.2.2 The system shall provide a visible "View Rules" control during gameplay.

5.2.2.3 The system shall display the rules when the "View Rules" control is activated.

Rules are embedded within each Game Module and do not require a network request.

5.2.3 Gameplay Interaction

The system must support interactive gameplay and enforce game rules. All state updates happen synchronously; the system completes each action before accepting the next input. The UI Renderer depends on the Game Module having returned a valid state before repainting.

Requirements:

5.2.3.1 The system shall display the current game state at all times during gameplay.

5.2.3.2 The system shall accept user input corresponding to valid in-game actions.

5.2.3.3 The system shall update the displayed game state after each valid user action.

5.2.3.4 The system shall reject any user action that violates the rules of the active game.

When an action is rejected, the system will display a visual indicator or brief message explaining why.

5.2.3.5 The system shall prevent modification of the game state after a rejected action.

5.2.3.6 The system shall determine when the active game has reached a terminal state.

5.2.3.7 The system shall display the outcome of the game upon reaching a terminal state.

5.2.3.8 The system shall disable further gameplay interaction after the game outcome is displayed.

The user will be shown an option to return to the main menu after the outcome is displayed.

5.2.4 Multiplayer Support

The system must support multiple modes of play.

Requirements:

- 5.2.4.1** The system shall provide a single-player mode for each board game included in the initial release.
- 5.2.4.2** The system shall provide a two-player local mode for each board game included in the initial release.
- 5.2.4.3** The system shall alternate turns between Player 1 and Player 2 during two-player mode.
- 5.2.4.4** The system shall visibly indicate the active player during two-player mode.

5.2.5 Computer Opponent Module

Certain games include a computer-controlled opponent. The AI Opponent Module depends on the active Game Module having a valid state available before any move is generated.

Requirements:

- 5.2.5.1** The system shall provide a computer-controlled opponent in single-player mode.
- 5.2.5.2** The system shall generate a valid move for the computer opponent during the computer's turn.

If no valid move can be generated, the system will treat this as a terminal condition.

- 5.2.5.3** The system shall update the game state after the computer opponent completes its move.
- 5.2.5.4** The system shall prevent the computer opponent from performing invalid moves.

Move validation for the computer opponent uses the same Rule Engine as human input.

5.2.6 Error Handling

The system must handle unexpected conditions without entering an undefined state.

Requirements:

- 5.2.6.1** The system shall display an error message if a game module fails to initialize.

The error message will direct the user back to the main menu.

- 5.2.6.2** The system shall ignore out-of-sequence user interactions (e.g., clicking game elements before the module has finished loading).

5.3 Performance Requirements

Performance requirements specify measurable operational criteria for the system under normal conditions.

5.3.1 Main Menu Load Time

5.3.1.1 The system shall load the main menu within 3 seconds on a broadband internet connection with a minimum speed of 25 Mbps.

5.3.2 Input Response Time

5.3.2.1 The system shall respond to user input within 100 milliseconds under normal operating conditions.

Response time is measured from user input event detection to visible UI update.

5.3.3 AI Decision Time

5.3.3.1 The system shall complete AI move generation and apply a move within 2 seconds of the start of the computer opponent's turn.

If the AI cannot complete within 2 seconds, it will fall back to selecting a random valid move.

5.3.4 Memory Usage

5.3.4.1 The system shall not exceed 150 MB of browser memory during a 30-minute game session under normal gameplay conditions.

5.3.5 Runtime Stability

5.3.5.1 The system shall operate continuously for at least 30 minutes of uninterrupted gameplay without crashing or freezing.

5.3.6 Network Usage After Load

5.3.6.1 The system shall make no additional network requests during an active game session after the initial page load.

5.4 Environment Requirements

This section specifies hardware and software resources required for development and execution of the system.

5.4.1 Development Environment Requirements

The system shall be developed using the following technologies:

- HTML
- CSS
- JavaScript
- Git for version control

Requirements:

5.4.1.1 The development environment shall support execution of JavaScript.

5.4.1.2 The development environment shall support deployment to a static web hosting service.

5.4.2 Execution Environment Requirements

Hardware Requirements:

Category	Requirement
Device	Desktop or laptop computer
Input	Keyboard and mouse or trackpad
Display	Minimum resolution of 1280 × 720 pixels

Software Requirements:

Category	Requirement
Browser	Modern web browser (Chrome, Firefox, Safari, or Edge)
Operating System	Any OS capable of running a supported browser (Windows, macOS, Linux)
Network	Active internet connection for initial page load
Browser APIs	DOM, localStorage, requestAnimationFrame
Installation	None required

Requirements:

5.4.2.1 The system shall execute entirely within a web browser.

5.4.2.2 The system shall not require installation of additional software.

5.4.2.3 The system shall function on the latest stable versions of Chrome, Firefox, Safari, and Edge.

5.4.2.4 The system shall use localStorage for any client-side persistent data, such as daily puzzle completion status.

If localStorage is unavailable, features that depend on it will be disabled gracefully.